

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON (NIVEL 1
(PRINCIPIANTES))

Clase 02: Variables, Tipos de Datos y Funciones

«El Almacén de Cajas en Memoria y la Licuadora Tipada»

*Dominio de la memoria Heap en Python: tipos primitivos (str, int, float, bool), paso de
parámetros a funciones con Type Hints (PEP 484) y casting.*

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Principiante Absoluto | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: El Almacén de Datos y la Memoria de la Computadora	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Ciclo de Transformación, Casting y Retorno	Pág. 5
Capítulo 3	Implementación Práctica en Python: Calculadora Financiera Modular con Type Hints	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Trampas Clásicas con Variables y Casting	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la asignación por referencia en el Heap de memoria, inmutabilidad y contratos de funciones con Type Hints.



Competencia Práctica

Construir funciones modulares tipadas que soliciten datos, ejecuten conversiones seguras y devuelvan resultados formateados.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. El Almacén de Datos y la Memoria de la Computadora

En Python, las variables no contienen datos directamente; son identificadores que apuntan a objetos en la memoria Heap.

☀ Metáfora Central: El Almacén de Cajas en Memoria y la Licuadora Tipada

Imagina un almacén con cajas etiquetadas (variables) y una licuadora modular (funciones). La licuadora recibe ingredientes tipados (parámetros str, float, int), procesa la mezcla mediante casting y entrega una nueva caja con el resultado.

Principios Teóricos y Modelo Mental

Python es fuertemente tipado: no realiza conversiones implícitas incompatibles. Los tipos primitivos (int, float, str, bool) son inmutables.

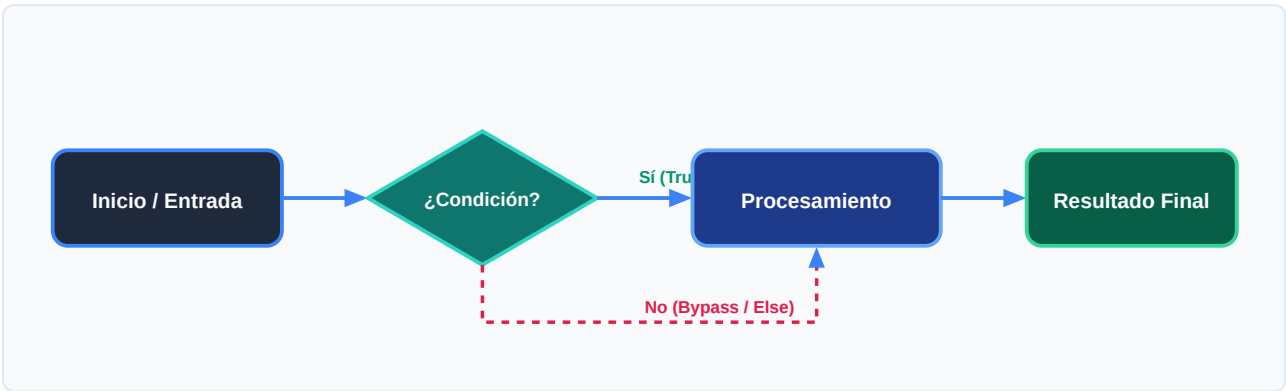
Al conectar variables con funciones tipadas (PEP 484), creamos transformaciones seguras, documentadas y fácilmente depurables.

⚡ Regla de Oro en Python

Convierte tipos explícitamente con `int()` o `float()` y anota siempre los tipos de parámetros y retornos en tus funciones.

2. Ciclo de Transformación, Casting y Retorno

Flujo de recepción de datos textuales, paso de referencias a funciones, casting numérico y retorno en Heap.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	La función input() captura la entrada del usuario como string en memoria.	Buffer de entrada -> '45.90' (str)
2. Evaluación	Paso de referencia a la función modular calcular_precio_total().	Parámetros locales reciben punteros del Heap
3. Transformación	Casting explícito float() e int() y cálculo matemático con impuestos.	Nuevo objeto float instanciado en Heap
4. Retorno / Salida	return entrega el resultado y f-string proyecta el resumen en pantalla.	Render formateado en consola

Visualización Mental

Siempre encapsula tus operaciones de cálculo y casting en funciones puras para facilitar la verificación con pruebas unitarias.

3. Calculadora Financiera Modular con Type Hints

Programa modular que define funciones con anotaciones de tipo, realiza casting seguro y formatea con f-strings:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
# 1. Función Modular con Type Hints (PEP 484)
def calcular_total_pedido(precio_str: str, cantidad_str: str, tasa_iva: float = 0.21) -> float:
    """Castea entradas de texto a float/int y calcula el monto total con IVA."""
    precio: float = float(precio_str)
    unidades: int = int(cantidad_str)
    subtotal: float = precio * unidades
    total_con_iva: float = subtotal * (1.0 + tasa_iva)
    return round(total_con_iva, 2)

# 2. Invocación del flujo de cálculo
precio_input: str = "45.90"
cantidad_input: str = "3"
total_pagar: float = calcular_total_pedido(precio_input, cantidad_input)

print(f"Total a Pagar: ${total_pagar:,.2f}")
```

Análisis del Código Fuente

La función declara contratos estrictos de tipo (`precio_str: str -> float`), realiza casting explícito y devuelve un valor redondeado listo para producción.

4. Trampas Clásicas con Variables y Casting

Errores comunes de principiantes al trabajar con tipos de datos y parámetros:

⚠ Gotcha Frecuente (Trampa de Principiante)

Intentar sumar directamente cadenas con números ('10' + 5) generando TypeError, o no convertir entradas antes de operar.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
precio = input('Precio: ')
total = precio * 2 # Concatena: '1010'
```

✓ Patrón Pythonic / Correcto

```
def calcular(precio_str: str) -> float:
    return float(precio_str) * 2 # Operación
numérica real
```

🛡 Consejo de Resiliencia en Producción

Define siempre funciones con anotaciones de tipo explícitas para prevenir fallos silenciosos en tiempo de ejecución.

5. Resumen Ejecutivo y Conclusiones

Dominas los tipos primitivos esenciales de Python, la inmutabilidad en memoria Heap y la modularización en funciones con Type Hints.



Logro Alcanzado en esta Clase

Capacidad para construir funciones robustas con casting seguro y cálculos matemáticos testeados.

Notas del Instructor y Sigüientes Pasos

En la siguiente clase exploraremos las compuertas lógicas y condicionales (if/elif/else): cómo dotar a la computadora de toma de decisiones.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Implementa una función `calcular_propina(total_cuenta: float, porcentaje: float) -> float` y pruébala con `pytest`.